Hackathon Prompt:
Quantum Random Number Generator

Background and motivation:

Random numbers are everywhere in computing. They are used in simulations, games, scientific research, cryptography, security, and many other applications.

But there is an important distinction between **pseudo-random** and **true random** numbers.

Most classical computers generate pseudo-random numbers using mathematical algorithms. These numbers may look random, but if you know the algorithm and its starting conditions, the sequence can theoretically be reproduced.

Quantum mechanics gives us a fundamentally different source of randomness. When we measure a quantum system in superposition, the outcome is inherently probabilistic.

Your challenge is to use **Qiskit** to build a quantum random number generator.

Getting started:

Create a quantum circuit that generates a random number between **0** and **n**.

You are free to choose the range of numbers your generator supports.

Start by asking yourself:

- How many possible states can I represent with one qubit?
- How many can I represent with two?
- What about three?

Use **Hadamard gates (H)** to put your qubits into superposition.

Your circuit should:

1. Create the required number of qubits.
2. Put the qubits into an equal superposition.
3. Measure the qubits.
4. Convert the measured binary state into a numerical value.
5. Return the value as your random number.

Tips:

You can control how many times your circuit is executed using **shots.**

For example, if you run your circuit 1,000 times, you should expect the possible states to appear approximately equally often.

Try increasing the number of shots and observe what happens.

Ask yourself:

- Does 10 shots give the same distribution as 10,000 shots?

Getting started:

As you experiment, you may notice that some outcomes appear slightly more frequently than others.

Why?

Is the generator actually biased, or could this simply be statistical variation?

If you run the circuit on real quantum hardware, you may also encounter physical noise.

Investigate:

- How does noise affect your random number generator?
- Does the distribution remain uniform?
- How could you detect bias?
- Can you apply error mitigation?
- How many shots are needed before the distribution becomes statistically convincing?

Stretch Challenge:

Build a function:

`quantum_random(min, max)`

that allows the user to request a random integer within any specified range.